MVP 2 is the backend—the central record connecting your hardware, owner dashboard, Nemotron agent, and Photon communications.

Its most important job is to keep the workflow correct:

Receive readings → track an incident → support investigation → record owner approval → authorize technician contact → track the repair → preserve useful experience.

The backend should work with sample data before the hardware, agent, and messaging integrations are finished.

1. Scope and technology

Item	Choice
Runtime	Node.js
Application framework	Express
Language	JavaScript
Persistent storage	One local JSON file
Recent sensor history	Bounded arrays in memory
Frontend updates	Frontend requests state every two seconds
Hardware connection	Receives HTTP requests from the UNO Q adapter
AI integration	Tool functions used by Nemotron
Communications integration	Functions used by the Photon integration

Use **one running backend process** for the hackathon. A single process makes JSON-file storage and coordination much simpler.

MVP 2 does not implement the dashboard, wire sensors, or build the voice pipeline. It provides the records and controlled actions those components need.

2. Current hardware configuration the backend understands

There is **one UNO Q controlling two building models**.

Building ID	Building	Sensor	Outputs

`office`	Office	Grove light sensor	Three white LEDs and one servo representing a fan
`data_center`	Data Center	Flame/IR sensor	Three green LEDs and one servo representing a fan

Temperature sensors are not required until confirmed.

Do not include humidity, water, door, tap, or measured fan-speed data in the current schema.

The backend must distinguish:

- **Measured readings:** light level and flame/IR signal.
- **Requested commands:** what the application wants the LEDs or servos to do.
- **Acknowledged commands:** what the controller reports accepting.
- **Physical results:** only what the available sensors or a technician actually confirm.

3. Core records

Keep these collections in the persistent JSON file:

Collection	What it contains
Buildings	Names, types, equipment inventory, and operating expectations
Device configuration	UNO Q identity, sensor mappings, calibration, and input interpretation
Incidents	Evidence, investigation findings, approval links, and resolution
Service requests	Proposed work awaiting the owner's decision
Technicians	Demo contact identifiers, skills, and availability
Work orders	Approved assignments, contact attempts, progress, and findings
Repair cases	Confirmed or explicitly simulated maintenance experience
Activity events	Owner-readable history of important changes
Pending hardware commands	Commands awaiting controller acknowledgment

Keep recent continuous readings in memory. Save important reading snapshots with incidents and repairs so the evidence survives a restart.

4. Receive and validate hardware readings

The UNO Q adapter sends structured snapshots to the backend.

Every snapshot should identify:

- Schema version.
- Device ID.
- Sequence number.
- Microcontroller uptime.
- Building IDs.
- Sensor values.
- Commanded equipment states.
- Whether the data is hardware or simulated.

The backend adds its own **received-at timestamp**.

Validation should check:

Check	Required behavior
Known device	Accept the configured UNO Q identity.
Known building	Accept `office` and `data_center`.
Correct value type	Light readings must be finite numbers; normalized IR alert must be a boolean.
Plausible bounds	Check against the configured ADC range where known.
Duplicate snapshot	Avoid adding duplicate events or readings.
Restart	Allow sequence numbers to reset after a detected controller restart.
Malformed payload	Reject it with a clear error without crashing.

Ideally, add a `boot_id` to the hardware contract so sequence numbers are unambiguous across restarts. Agree on that with the MVP 1 developer.

Do not assume a light-sensor ADC range of 0–1023 or 0–4095 unless that configuration is explicitly known.

5. Recent readings and connection status

Maintain a bounded history for each sensor. For the demo, around **ten minutes of readings** is enough.

Each stored reading needs:

Field	Purpose
Device and building IDs	Identify its source.
Sensor ID	Identify the measurement.
Raw value	Preserve the original evidence.
Received timestamp	Determine freshness.
Sequence/boot information	Handle duplicates and restarts.
Source	Hardware or simulated.

Use a configurable freshness timeout. If the adapter normally reports every second, five seconds without a snapshot is a reasonable starting value to test.

The backend should expose:

- Fresh readings.
- Stale readings.
- Device unavailable.
- No readings received yet.

Fresh readings do not prove every sensor is physically connected. Analog inputs can float when disconnected, and digital inputs can remain at a plausible level. Do not invent automatic sensor-health confirmation.

Because both buildings share one UNO Q, a controller connection failure affects both.

6. Detect an issue without flooding the system

Use simple, calibrated rules to start investigations. Nemotron performs the subsequent reasoning.

For the office lighting scenario:

1. Check that the application expects the office lights to be on.

2. Read the current light level and recent history.
3. Compare it with the calibrated baseline for that lighting condition.
4. Require the abnormal condition to persist for a configured period.
5. Create or update an incident.

Avoid treating intentionally switched-off lights as a failure.

For the data-center IR scenario:

1. Read the normalized IR signal.
2. Preserve short activations reported by the adapter.
3. Record the evidence.
4. Create or update an **IR alert investigation**.

Do not label that event a confirmed fire.

Deduplication is essential: repeated readings from the same unresolved condition should update one incident. They must not create a new ticket, owner text, or technician call every reporting interval.

Use a simple active-incident key such as:

building + sensor + issue category

The detection and recovery conditions should tolerate normal noise. For lighting, require sustained recovery rather than closing the incident after one normal sample.

7. Keep incident status separate from service status

An owner declining a visit does not mean the physical issue disappeared.

Use separate fields:

Field	Suggested values
Incident status	Open, awaiting verification, resolved
Investigation status	Pending, running, complete, failed
Approval status	Not requested, pending, approved, declined, expired
Work-order status	Draft, ready for contact, awaiting acceptance, accepted, in progress, repair reported, closed, needs attention

The frontend can translate these into simple owner-facing labels.

For example:

- Incident: **Open**
- Approval: **Declined**
- Work order: **Not dispatched**

That correctly means the owner declined service while the issue remains unresolved.

8. Support Nemotron through tools

Provide backend functions that the agent can call.

Function	Returns or changes
`getSensorHistory`	Recent readings, freshness, and source
`getBuildingProfile`	Building type, requirements, and equipment
`getCommandedStates`	Requested and acknowledged LED/servo states
`searchRepairCases`	Relevant confirmed cases with supporting metadata
`getAvailableTechnicians`	Candidates filtered by skills and availability
`createServiceRequest`	A proposed inspection or repair request
`getApprovalStatus`	The recorded owner decision
`requestTechnicianContact`	An authorized communication action
`recordFindings`	Technician-reported inspection and repair information
`requestVerification`	Start or update the recovery check

These can be ordinary JavaScript service functions. If the Nemotron integration runs in the same Node process, it can import them directly. Separate HTTP wrappers are only needed where another process needs access.

Do not pass the entire database to the model on every request. Return the relevant records and enough evidence to explain them.

The backend supplies facts and enforces permissions. Nemotron chooses what evidence to request and what response to propose.

9. Owner approval is a backend-enforced gate

A service request should contain:

- Request ID and incident ID.
- Building.
- Proposed action.
- Reason and supporting evidence.
- Uncertainty.
- Creation time.
- Proposal version.
- Approval status.
- Recorded owner decision and time.

The owner can approve through:

- The dashboard.
- A Photon message response associated with that request.

Both routes must call the **same decision function**.

Before accepting approval, check:

1. The request exists.
2. It is still awaiting a decision.
3. The sender/session belongs to the configured owner.
4. The decision applies to the current proposal.
5. The request has not already been superseded.

A bare “yes” should not authorize an arbitrary pending request. Associate replies with the relevant conversation and request, or require a short request identifier.

If the proposed work changes materially after approval, create a revised request rather than silently reusing the old approval.

No response means pending.

10. Authorize and track Photon actions

The backend should create communication actions for MVP 5 to execute.

Action	When allowed
Clarification text to owner	Relevant to an active investigation
Approval-request text	A service request is ready

Technician call	The corresponding service request is approved
Owner progress update	There is a recorded service event to communicate

For each action, save:

- Action ID.
- Incident/work-order ID.
- Recipient identifier.
- Channel.
- Purpose.
- Pending/sent/failed/unknown status.
- Provider reference, when available.
- Attempt time and result.

Reserve the technician-contact action before making the external request. Repeated owner approvals should find that existing action rather than create another one.

If the provider's response is ambiguous—for example, a timeout after the call may already have started—mark the outcome unknown and check it before retrying. Don't blindly place another call.

A completed call is not technician acceptance. Acceptance must come from an explicit response captured by the communications integration.

11. Hardware commands

The backend stores commands for the UNO Q adapter to retrieve.

Required command types:

Type	Parameters
Set LED	Building ID, LED number, on/off
Set fan representation	Building ID, running/stopped

Every command needs:

- Unique command ID.
- Target device.
- Target building.
- Allowed action and parameters.

- Creation time.
- Pending/acknowledged/rejected status.

Reject unknown buildings, invalid LED numbers, and unsupported actions.

Store **requested state** separately from **controller-acknowledged state**. Neither is a measured servo position.

Do not automatically start a fan representation as a response to an IR alert. The hardware's demonstration actions should be explicitly defined.

12. Record repair experience correctly

A repair case should contain:

Information	Why it matters
Source building	Shows where the experience originated.
Equipment type/configuration	Establishes whether another case is comparable.
Symptoms and evidence	Supports meaningful retrieval.
Suspected cause	Preserves the original hypothesis.
Confirmed cause	Records the technician's finding separately.
Repair action	Explains what was done.
Outcome	Records whether it worked.
Verification method	Distinguishes sensor recovery from manual confirmation.
Source label	Real incident or synthetic demonstration case.

Do not convert an agent's guess into a confirmed cause.

Start retrieval with simple filters and text matching. A vector database is unnecessary for this version.

Useful filters include equipment type, issue category, confirmed status, and outcome. A case from another building should be returned because it is relevant—not merely because it came from a different building.

13. Verification and closure

Track these events separately:

1. Technician reports completing work.
2. Findings are recorded.
3. Relevant readings are reviewed.
4. The outcome is verified.
5. The incident is resolved.

For office lighting, verification can include sustained readings returning to the calibrated range while lights are expected to be on.

For an IR alert, a clear signal only establishes that the signal is no longer active. It does not independently confirm the cause or prove fire safety.

If the available sensors cannot verify the repair, record a **manual verification with the verifier and reason**. Do not present it as sensor-confirmed recovery.

14. Proposed HTTP interface

These are our application routes, not provider endpoints.

Method and route	Consumer	Purpose
`POST /api/hardware/telemetry`	UNO Q adapter	Submit a snapshot.
`GET /api/hardware/commands`	UNO Q adapter	Retrieve pending commands.
`POST /api/hardware/commands/:id/ack`	UNO Q adapter	Acknowledge or reject a command.
`GET /api/state`	Frontend	Fetch the owner-dashboard summary.
`GET /api/buildings/:id/readings`	Frontend/agent	Retrieve recent readings.
`GET /api/incidents/:id`	Frontend/agent	Retrieve evidence, findings, and service state.
`POST /api/incidents/:id/investigate`	Frontend	Request a Nemotron investigation.

`POST /api/incidents/:id/service-request`	Agent	Create an approval proposal.
`POST /api/service-requests/:id/decision`	Owner interface	Record approval or decline.
`GET /api/activity`	Frontend	Retrieve the event timeline.
`GET /api/repair-cases`	Frontend/agent	Search relevant history.
`POST /api/work-orders/:id/findings`	Technician integration	Record inspection and repair findings.
`POST /api/incidents/:id/verify`	Authorized workflow	Record verification or explain why it remains pending.
`POST /api/hardware/commands`	Authorized demo controls	Request supported LED/servo actions.

Photon messages can enter through an SDK event handler that calls the same backend functions. A webhook endpoint is only necessary if the selected Photon integration uses webhooks.

15. What `GET /api/state` should provide

Give the frontend a ready-to-display summary:

- Device connection and last update.
- Each building's latest readings.
- Owner-friendly building status.
- Open incidents.
- Pending approval requests.
- Current work-order progress.
- Requested and acknowledged equipment states.
- Whether displayed data is live or simulated.

Keep internal provider credentials, private integration details, and unnecessary technician contact information out of this response.

The owner should not need to interpret database records or agent logs to understand what is happening.

16. Activity history

Record meaningful transitions:

- Issue detected.
- Investigation started/completed/failed.
- Owner clarification requested.
- Service request created.
- Approval text sent or failed.
- Owner approved/declined.
- Technician contact attempted.
- Technician accepted/declined.
- Repair findings submitted.
- Recovery verified.
- Incident resolved.
- Device connection lost/restored.

Continuous sensor readings belong in Sensor Data, not as individual Activity entries.

17. Persistence and restart behavior

For JSON storage:

1. Load and validate the file at startup.
2. Serialize mutations through one write queue.
3. Write a temporary file.
4. Replace the saved file after a successful write.
5. Keep a small recoverable backup.
6. Preserve IDs and workflow state across restarts.

Do not silently replace an unreadable file with an empty database. Report the problem and preserve the original file.

On restart:

- Sensor state starts unknown until fresh readings arrive.
- Open incidents and approvals remain.
- Completed actions remain completed.
- In-progress external calls with uncertain outcomes require checking rather than automatic duplication.
- Recent in-memory charts may start empty; incident evidence remains saved.

18. Configuration and access

Keep configuration outside the frontend:

- Owner contact identifier.
- Technician demo contacts.
- Device authentication token.
- Allowed frontend origin.
- Detection thresholds and calibration.
- Freshness timeout.
- Nemotron and Photon credentials, when integrations are added.

Use a simple protected owner session for dashboard decisions and a separate token for the device adapter. Do not trust an `owner: true` field supplied in a request body.

Keep provider credentials out of the JSON records, source control, frontend bundle, and logs.

19. Testing MVP 2

Test	Expected result
Valid hardware snapshot	Correct building readings update.
Malformed snapshot	Clear rejection; server stays running.
Duplicate snapshot	No duplicate event or incident.
Controller restart	New readings are accepted correctly.
Reporting stops	Both buildings show unavailable/stale device data.
Office lights intentionally commanded off	No unexpected-darkness service request.
Persistent unexpected darkness	One incident is created.
Repeated abnormal readings	Existing incident updates; no message flood.
Owner declines	Technician contact is blocked; issue remains open.
Owner approves twice	One technician-contact action exists.
Wrong sender or request ID	Decision is rejected.
Call result is uncertain	No blind duplicate call.
Technician reports completion	Incident awaits verification.
Verified repair saved	Relevant retrieval can find it.

Unrelated case searched	It is not presented as diagnostic evidence.
Backend restarts	Persistent workflow state survives.
Simulated data used	Its source remains visible.

Use fixtures and fake communication results to test the backend first. The full integrations are added by the Nemotron and Photon MVPs.

20. Handoff deliverables and definition of done

The backend developer should provide:

- Running Express server.
- Documented routes and shared payload formats.
- Seed building, equipment, technician, and labeled repair records.
- Telemetry validation and bounded reading history.
- Incident deduplication.
- Approval enforcement.
- Hardware command queue and acknowledgments.
- Communication-action tracking.
- Repair verification and history storage.
- Restart-safe JSON persistence.
- Sample requests and meaningful workflow tests.
- An environment-variable example containing no real secrets.

MVP 2 is complete when sample hardware data can create one incident, support an investigation, obtain a recorded owner decision, authorize exactly one intended technician-contact action, track findings, and save a verified repair—while giving the frontend a clear view of every stage.

The actual external call may still be mocked at this stage. It becomes a live Photon action when MVP 5 is connected.